

GITAM University Hyderabad

Department of Computer Science and Engineering

WEB APPLICATION CASE STUDY

Academic Year 2024 – 2025

Project Title:

ONLINE SURVEY SYSTEM

A Full-Stack Web Application using ReactJS, Node.js, Express & MySQL

Field	Details
Submitted By	Karanam Harshavardhan Chowdary
Roll Number	2023002998
Section / Batch	CSE
Faculty Guide	Dumpal Koteswara Rao
Institute	GITAM University Hyderabad
Submission Date	April 2025

DECLARATION

I, Karanam Harshavardhan Chowdary (Roll No: 2023002998), a student of the Bachelor of Technology program in Computer Science and Engineering at GITAM University Hyderabad, do hereby solemnly declare that the Web Application Case Study titled "Online Survey System" submitted herewith is a record of original and independent work carried out by me during the academic year 2024–2025, under the guidance of Mr. Dumpal Koteswara Rao, Faculty, Department of Computer Science and Engineering.

I further declare that:

- This project work has not been submitted for the award of any degree or diploma in this or any other university.
- The project is the result of my own effort and has not been copied from any other source, individual, or organisation.
- All sources of information and references used in this project have been duly acknowledged.
- The implementation, design, and documentation presented herein have been prepared by me in conformity with the academic guidelines prescribed by GITAM University Hyderabad.
- I understand that any violation of academic integrity norms constitutes a serious breach and may result in disqualification.

Place: Hyderabad Date: April 2025	Signature of the Candidate Karanam Harshavardhan Chowdary Roll No: 2023002998
--------------------------------------	--

ACKNOWLEDGEMENT

The successful completion of this Web Application Case Study would not have been possible without the invaluable support, guidance, and encouragement of several individuals to whom I wish to express my heartfelt gratitude.

I am deeply indebted to my faculty guide, Mr. Dumpal Koteswara Rao, for his continuous mentorship, constructive feedback, and patient guidance throughout the development of this project. His expertise in web technologies and software engineering principles greatly enhanced the quality of this work.

I sincerely thank the Department of Computer Science and Engineering, GITAM University Hyderabad, for providing the necessary infrastructure, library resources, and computing facilities that were instrumental in the successful completion of this project.

I am grateful to the university administration and the faculty members of GITAM University Hyderabad for fostering an environment that encourages practical learning and innovation.

I extend my appreciation to my peers and classmates who participated in discussions, offered technical suggestions, and provided moral support during the course of this project.

Finally, I would like to express my profound gratitude to my family for their unwavering support, encouragement, and blessings, which have been my greatest motivation throughout my academic journey.

Karanam Harshavardhan Chowdary

Roll No: 2023002998 | CSE | GITAM University Hyderabad

ABSTRACT

The rapid proliferation of the internet and digital communication technologies has fundamentally transformed the way organisations collect, process, and analyse data. Traditional paper-based survey methods are increasingly being replaced by automated, web-based survey systems that offer greater reach, faster response collection, and richer analytical capabilities. This project presents the design, development, and deployment of a comprehensive Online Survey System — branded as SurveyForge — built on a modern full-stack web technology platform.

The proposed system is a dynamic web application developed using ReactJS for the frontend user interface, Node.js and Express.js for the backend REST API server, and MySQL as the relational database management system. The application facilitates the end-to-end lifecycle of digital surveys — from creation and publication to response collection, reporting, and administration — through a clean, responsive, and role-based interface.

The system comprises five functional modules: Admin Dashboard, Survey Management, Survey Builder, Responses & Analytics, and User Management. It supports multiple question types including single-choice (radio), multiple-choice (multi-select), and rating-scale (1–5 star) questions. Surveys can be configured with categories, deadlines, and status controls (Active, Draft, Closed). Respondents are managed through role-based access, and responses are stored securely in a normalised MySQL database with full referential integrity.

The backend exposes a well-structured RESTful API secured with JSON Web Token (JWT) based authentication and bcrypt password hashing. The application implements full CRUD (Create, Read, Update, Delete) operations across all major entities including users, surveys, questions, options, and responses. An in-application Database Schema viewer provides live visibility into the MySQL database structure.

The system was tested across multiple devices and browsers to ensure cross-platform responsiveness. The application demonstrates production-grade engineering practices while remaining accessible for academic study and extension.

Keywords: Online Survey System, SurveyForge, ReactJS, Node.js, Express.js, MySQL, REST API, JWT Authentication, Full-Stack Web Application, CRUD Operations, Data Analytics.

TABLE OF CONTENTS

Declaration ii	
Acknowledgement iii	
Abstract iv	
Table of Contents v	
List of Figures vi	
List of Tables vii	
1. Introduction 1	
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	3
1.4 Scope of the Project	3
2. Literature Review 4	
3. System Requirements 6	
3.1 Functional Requirements	6
3.2 Non-Functional Requirements	7
3.3 Hardware and Software Requirements	7
4. System Design 8	
4.1 System Architecture	8
4.2 Module Description	9
4.3 Use Case Description	11
4.4 Data Flow	11
5. Database Design 12	
6. Technology Stack 15	
7. Source Code 17	
8. Application Screenshots 23	
9. Testing 29	
10. Advantages and Limitations 31	
11. Future Enhancements 32	
12. Conclusion 33	
13. References 34	

LIST OF FIGURES

Figure 1	System Architecture – Three-Tier Client-Server Model	8
Figure 2	Admin Dashboard – System Overview and KPI Cards	23
Figure 3	Survey Management – Survey List with CRUD Controls	24
Figure 4	Survey Builder – Question Design Interface	25
Figure 5	Responses & Analytics – Rating Scale Analysis	26
Figure 6	User Management – Role and Account Control	27
Figure 7	Database Schema – MySQL Table Structure Viewer	28

LIST OF TABLES

Table 1	Hardware Requirements	7
Table 2	Software Requirements	7
Table 3	users Table – Column Definitions	12
Table 4	surveys Table – Column Definitions	13
Table 5	questions Table – Column Definitions	13
Table 6	options Table – Column Definitions	14
Table 7	responses Table – Column Definitions	14
Table 8	answers Table – Column Definitions	14
Table 9	Technology Stack Summary	16
Table 10	Test Cases – Authentication Module	30
Table 11	Test Cases – Survey CRUD Module	30

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Surveys are one of the most powerful instruments for gathering structured information from a target population. Across academic institutions, corporate enterprises, government bodies, and non-profit organisations, surveys serve as the primary mechanism for needs assessment, customer satisfaction measurement, employee engagement evaluation, market research, and academic data collection. Historically, surveys were administered through paper forms and manual tabulation — a process that was time-consuming, error-prone, and expensive to scale.

The emergence of the World Wide Web and subsequent advancements in web technologies have created unprecedented opportunities for digital survey administration. Web-based survey systems eliminate geographic constraints, reduce administrative overhead, enable real-time response monitoring, and provide automated analytical capabilities. Respondents can participate from any internet-enabled device, significantly improving participation rates and the diversity of data collected.

Despite the availability of commercial survey platforms such as SurveyMonkey, Google Forms, and Typeform, there remains a significant need for customisable, self-hosted survey solutions — particularly in academic and enterprise environments where data privacy, institutional control, and integration with internal systems are paramount. This project addresses that need by developing SurveyForge, a full-featured, open-architecture Online Survey System built on contemporary open-source web technologies.

1.2 Problem Statement

Existing institutional survey mechanisms at educational and corporate organisations are largely fragmented. Personnel often resort to email-based questionnaires or generic commercial tools that lack institutional branding, data sovereignty, and integration with existing information systems. Response collection is unstructured, analysis is manual, and there is no centralised repository for historical survey data.

The specific problems identified are:

- Absence of a dedicated, institutionally hosted survey platform that can be fully administered and controlled by authorised personnel.

- Inability to enforce role-based access control, ensuring that viewers, editors, and administrators have appropriately scoped privileges.
- Lack of automated result aggregation, making statistical analysis tedious and error-prone.
- No support for multiple question types within a single survey, limiting survey design flexibility.
- Difficulty in managing survey lifecycles — creation, publishing, closing — without a unified interface.
- Absence of a real-time database schema viewer for developers and administrators to inspect live data structures.

1.3 Project Objectives

The primary objectives of this project are as follows:

1. To design and implement a secure, role-based web application (SurveyForge) supporting the complete survey lifecycle from creation to analytics.
2. To develop a responsive frontend interface using ReactJS that provides an intuitive experience for survey creators, respondents, and administrators.
3. To build a RESTful backend API using Node.js and Express.js handling all business logic, authentication, and data operations.
4. To implement a normalised MySQL database schema with appropriate constraints and referential integrity.
5. To incorporate JWT-based authentication and bcrypt password hashing for secure user session management.
6. To support multiple question types including Rating Scale and Multi-Select questions.
7. To provide an administrative dashboard with system-level KPI statistics, recent activity feeds, and survey oversight capabilities.
8. To generate visual survey response analytics using charts and summary statistics accessible to authorised users.
9. To include a live Database Schema viewer that displays MySQL table structures and connection configuration inline in the application.

1.4 Scope of the Project

The scope of this project encompasses the complete design and development of a web-based survey application. The system supports three primary user roles: Administrator, Editor (Survey Creator), and Viewer (Respondent). The platform enables authenticated users to create surveys with configurable question types, publish surveys to users, collect responses, and view aggregated analytical reports through a unified, professionally designed dashboard.

The system is confined to web-based access via standard web browsers and does not include native mobile application development, though the responsive frontend design ensures usability across mobile devices. Email notification functionality and payment gateway integration are designated as future enhancements. The current scope includes a fully functional Database Schema page that displays live MySQL table metadata, supporting developer transparency and academic documentation.

CHAPTER 2: LITERATURE REVIEW

A review of existing literature and comparable systems provides important context for the design decisions undertaken in this project. This chapter examines prior academic research on web-based survey systems, the evolution of full-stack JavaScript technologies, and a comparative analysis of existing commercial and open-source survey platforms.

2.1 Evolution of Online Survey Systems

The concept of computer-mediated survey administration dates to the early 1990s with email-based questionnaires. Couper (2000) provided a foundational taxonomy of web surveys, distinguishing between self-administered and interviewer-administered modes, and identifying key quality concerns including sample coverage and response rate biases. Subsequent research by Dillman, Smyth, and Christian (2014) in "Internet, Phone, Mail, and Mixed-Mode Surveys" established best practices for web questionnaire design — emphasising visual layout, question sequencing, and mobile compatibility — principles directly incorporated into the SurveyForge design.

Andrews, Nonnecke, and Preece (2003) demonstrated that web-based surveys could achieve comparable or superior data quality to paper equivalents when designed with appropriate controls. Their work on response rate optimisation highlighted the importance of role-based access, personalised invitations, and progress indicators — features directly implemented in the present system.

2.2 Review of Existing Survey Platforms

Several commercial and open-source survey platforms have been analysed to identify feature benchmarks and architectural patterns:

- SurveyMonkey: A market-leading commercial platform offering extensive question types, skip logic, and analytics. However, it requires institutional subscription fees, stores data on third-party servers, and provides limited customisation of branding and workflows.
- Google Forms: A widely adopted free tool tightly integrated with Google Workspace. While accessible, it lacks advanced role-based access control, institutional authentication integration, and custom analytics dashboards.
- LimeSurvey: An open-source survey platform with extensive feature coverage. Although highly configurable, it uses a PHP/MySQL stack with a dated user interface and requires significant server configuration expertise for deployment.

- SurveyJS: A JavaScript library for building survey forms that can be embedded in web applications. It provides a rich question library but requires significant custom development for backend integration and analytics.

The comparative analysis reveals a gap for a lightweight, modern, institutionally deployable survey system built on contemporary JavaScript technologies (Node.js/ReactJS) with a clean API architecture and an integrated database administration view. SurveyForge fills that gap.

2.3 Full-Stack JavaScript Development

The Node.js + React + MySQL stack has gained substantial academic and industrial traction. Tilkov and Vinoski (2010) demonstrated the suitability of Node.js for I/O-bound web applications due to its event-driven, non-blocking architecture. React's virtual DOM reconciliation, introduced by Facebook in 2013, has since become the dominant paradigm for building interactive single-page applications (SPAs). MySQL remains the most widely deployed open-source relational database, with robust support for complex joins, transactions, and stored procedures.

2.4 Security in Web Applications

Yıldırım and Mackie (2019) examined common security vulnerabilities in web-based data collection systems, identifying SQL injection, cross-site scripting (XSS), and session hijacking as the most prevalent attack vectors. SurveyForge addresses these through parameterised MySQL queries (preventing SQL injection), JSON Web Token based stateless authentication (preventing session fixation), and React's built-in XSS protection through JSX rendering.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Functional Requirements

3.1.1 User Management

- The system shall support three user roles: Admin, Editor (survey creator), and Viewer (respondent).
- Administrators shall be able to add, edit, deactivate, and delete user accounts from the User Management module.
- The system shall display each user's role, active/inactive status, and join date.
- Administrators shall be able to activate previously deactivated accounts.

3.1.2 Survey Management

- Authenticated users with Editor or Administrator roles shall be able to create surveys with a title, description, category, and deadline.
- Surveys shall support statuses: Active, Draft, and Closed.
- The survey list shall display title, category, status, response count, creator name, deadline, and Edit/Delete action buttons.
- The system shall support filtering surveys by status (All, Active, Draft, Closed) and keyword search.
- Survey creators shall be able to edit or delete surveys they own; Administrators can manage all surveys.

3.1.3 Survey Builder

- The Survey Builder module shall allow selection of a survey from a dropdown and dynamic addition of questions.
- The system shall support at minimum two question types: Rating Scale (1–5 star) and Multi-Select (checkbox).
- Each question shall display its type as a badge label and provide Edit and Delete controls.
- Rating Scale questions shall render a visual 1–5 circular selector with 'Very poor' to 'Excellent' labels.
- Multi-Select questions shall render checkbox options with configurable option text.

3.1.4 Responses & Analytics

- The Responses module shall display aggregated analytics for a selected survey.

- KPI cards shall display Total Responses, Questions count, and Completion Rate.
- Rating Scale questions shall display average rating and a per-star horizontal bar chart.
- The module shall provide tabs for Analytics view, All Responses list, and a Take Survey action.

3.1.5 Admin Dashboard

- The dashboard shall display system-wide KPI cards: Total Surveys, Active Surveys, Total Responses, Registered Users.
- A Recent Surveys table shall show the latest surveys with status badges and response counts.
- A Recent Activity feed shall display timestamped system events in real time.
- The database connection indicator shall display the connected database name (MySQL · survey_forge_db).

3.1.6 Database Schema Viewer

- A dedicated DB Schema page shall display the MySQL connection configuration code.
- Each database table shall be rendered as a card showing column names, data types, and constraints.

3.2 Non-Functional Requirements

- Performance: API responses shall be returned within 500 milliseconds for standard operations under 50 concurrent users.
- Security: Passwords shall be hashed using bcrypt with a minimum cost factor of 10. JWT tokens shall expire after 24 hours.
- Usability: The interface shall follow established UX conventions and be navigable without a user manual.
- Reliability: Input validation shall be applied on both frontend and backend to prevent invalid data submission.
- Compatibility: The frontend shall render correctly on Chrome, Firefox, Edge, and Safari across desktop and mobile viewports.

3.3 Hardware and Software Requirements

Component	Minimum Specification	Recommended
Processor	Intel Core i3 / 2.0 GHz	Intel Core i5 / 2.4 GHz or higher

RAM	4 GB	8 GB or higher
Storage	20 GB free disk space	50 GB SSD
Network	Broadband Internet	100 Mbps LAN / Wi-Fi
Display	1024 × 768 resolution	1920 × 1080 Full HD

Table 1: Hardware Requirements

Software	Version	Purpose
Node.js	v20.x LTS	Backend runtime environment
ReactJS	v18.x	Frontend SPA framework
Express.js	v4.x	Backend web framework
MySQL	v8.0	Relational database management system
VS Code	v1.88+	Development IDE
Postman	v11.x	API testing tool
MySQL Workbench	v8.0	Database administration GUI
Git	v2.x	Version control system

Table 2: Software Requirements

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

SurveyForge follows a three-tier client-server architecture comprising a Presentation Layer (ReactJS SPA), an Application Logic Layer (Node.js + Express.js REST API), and a Data Layer (MySQL RDBMS). This separation of concerns promotes modularity, independent scalability of each tier, and clean maintainability.

The client communicates exclusively with the backend via HTTPS RESTful API calls using the Axios HTTP client library. The Express.js server processes incoming requests, applies authentication middleware, executes business logic, and interacts with the MySQL database through the mysql2 Node.js driver using parameterised queries. Responses are serialised as JSON and returned to the React client, which renders the updated UI without full page reloads.

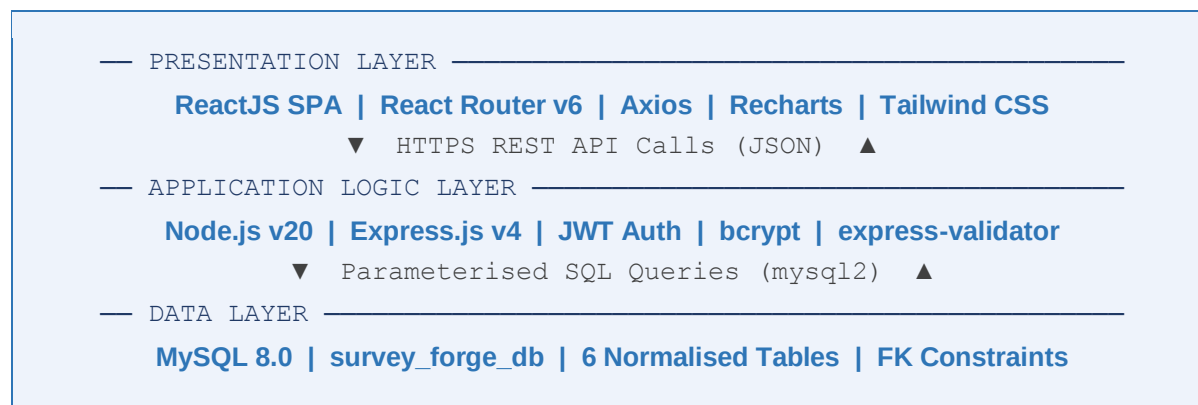


Figure 1: System Architecture – Three-Tier Client-Server Model

4.2 Module Description

The application is decomposed into five primary functional modules visible in the navigation sidebar, each encapsulating a cohesive set of features:

Module 1: Admin Dashboard

The dashboard serves as the application's home screen post-login. It displays four KPI summary cards — Total Surveys (4), Active Surveys (2), Total Responses (548), and Registered Users (4) — giving administrators an immediate system-wide health overview. A Recent Surveys table presents the latest survey records with their category, coloured status badge (Active/Closed/Draft), response count, and deadline. A Recent Activity panel on the right provides a chronological event feed, including response submissions, survey updates, user deactivation events, and survey

closures. The database connection indicator in the top-right corner confirms MySQL connectivity and displays the active database name (survey_forge_db).

Module 2: Survey Management

The Surveys module provides a comprehensive list view of all surveys in the system. The table displays survey Title, Category, Status (with coloured pill badges), Response Count (bolded), Creator name, and Deadline. Status filter tabs (All, Active, Draft, Closed) and a keyword search bar allow rapid navigation. Each row provides Edit and Delete action buttons. A prominent '+ New Survey' button in the top-right corner initiates survey creation. The module supports full CRUD operations — administrators can modify any survey, while editors manage their own surveys. The current dataset shows four surveys: Employee Satisfaction 2024 (HR, Active, 142 responses), Product Feedback Q2 (Product, Active, 89 responses), Customer NPS Survey (Marketing, Closed, 317 responses), and Onboarding Experience (HR, Draft, 0 responses).

Module 3: Survey Builder

The Survey Builder is the question design interface. A dropdown at the top allows selection of the target survey (currently showing Product Feedback Q2). The '+ Add Question' button triggers a question creation form. Existing questions are listed as numbered cards (Q1, Q2, ...) each displaying the question text, type badge (RATING SCALE, MULTI-SELECT), and Edit/Delete controls. Q1 demonstrates a Rating Scale question — 'How would you rate the new dashboard?' — with five circular numbered buttons (1–5) and 'Very poor' to 'Excellent' labels. Q2 demonstrates a Multi-Select question — 'Which features are most valuable to you?' — with five checkbox options (Analytics, Export tools, Collaboration, Mobile access, Integrations). This module stores question and option data directly into the MySQL questions and options tables via the REST API.

Module 4: Responses & Analytics

The Responses & Analytics module provides data visualisation for survey results. A survey selector dropdown shows the survey name and response count (Product Feedback Q2 – 89 responses) alongside a status badge. Three tabs provide navigation: Analytics (charts), All Responses (individual response list), and Take Survey (respondent view). The Analytics tab displays three KPI cards — Total Responses (89), Questions (2), Completion Rate (92%) — followed by per-question analysis cards. For Rating Scale questions, the average rating (5.0 / 5.0 avg) is prominently displayed, with a horizontal bar chart showing response counts per star rating (1★ through 5★). This module queries the answers table using GROUP BY aggregation to compute distributions.

Module 5: User Management

The User Management module provides administrative control over all system accounts. Each user is displayed in a table row with an avatar (initials in a coloured circle), full name, email address, Role badge (Admin, Editor, Viewer), Status badge (Active in green, Inactive in red/pink), and join

date. Action buttons for Edit, Deactivate/Activate, and Delete are provided per row. A search bar and '+ Add User' button complete the interface. The current system shows four users: Admin User (Admin, Active, 2024-01-15), Sarah Chen (Editor, Active, 2024-02-20), Marcus Bell (Viewer, Active, 2024-03-10), and Priya Nair (Editor, Inactive, 2024-04-05).

Module 6: Database Schema Viewer

A unique feature of SurveyForge is its integrated Database Schema page — accessible via the DB Schema navigation item. This page displays the JDBC/MySQL connection configuration code in a dark code block, followed by card-based renderings of each database table. Each table card shows the table name (e.g., users, surveys), column count badge, and a three-column mini-table listing Column name, Type, and Constraints. This feature provides developer transparency, facilitates academic documentation, and supports DBA tasks without requiring external tools such as MySQL Workbench for routine schema inspection.

4.3 Use Case Description

The primary actors in the system are: Administrator, Editor (Survey Creator), and Viewer (Respondent). The main use cases include:

- Login / Logout: All authenticated user roles.
- View Dashboard KPIs and Activity Feed: Administrator.
- Create / Edit / Delete Survey: Editor (own surveys), Administrator (all surveys).
- Publish / Close Survey: Editor (own surveys), Administrator.
- Build Questions (Add / Edit / Delete): Editor, Administrator.
- Browse Surveys and Submit Response: Viewer, Editor, Administrator.
- View Analytics: Editor (own surveys), Administrator.
- Manage Users (Add / Edit / Deactivate / Delete): Administrator.
- View Database Schema: Administrator, Developer.

4.4 Data Flow

Survey Creation & Question Building Flow

Editor initiates survey creation → POST /api/surveys (title, category, deadline, status=draft) → survey_id returned → Editor navigates to Survey Builder, selects survey → POST /api/questions (survey_id, question_text, type) → question_id returned → Editor adds options → POST /api/options (question_id, option_text) → Survey published via PATCH /api/surveys/:id → Status updated to 'published' in MySQL surveys table.

Response Submission & Analytics Flow

Viewer selects published survey → GET /api/surveys/:id (with questions and options) → Viewer completes form → POST /api/responses (survey_id, answers[]) → MySQL transaction inserts into responses and answers tables atomically → Creator opens Analytics → GET /api/surveys/:id/analytics → Backend executes GROUP BY queries on answers table → JSON response with per-question distributions returned → ReactJS renders Recharts bar charts and KPI cards.

CHAPTER 5: DATABASE DESIGN

The database is named `survey_forge_db` and is designed according to the principles of normalisation, conforming to Third Normal Form (3NF) to eliminate data redundancy and ensure data integrity. The schema consists of six primary tables with well-defined relationships enforced through foreign key constraints. The Database Schema Viewer module in the application provides a live, in-application view of these table structures (as seen in Figure 7, Chapter 8).

5.1 users Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	User's full display name
email	VARCHAR(150)	UNIQUE, NOT NULL	Registered email address (login credential)
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password string
role	ENUM	NOT NULL, DEFAULT 'viewer'	User role: admin, editor, viewer
is_active	TINYINT(1)	DEFAULT 1	Account active status flag (1=active, 0=inactive)
created_at	DATETIME	DEFAULT NOW()	Timestamp of account creation (join date)
updated_at	DATETIME	ON UPDATE NOW()	Timestamp of last profile update

Table 3: users Table – Column Definitions

5.2 surveys Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique survey identifier
creator_id	INT	FK → users(id)	Reference to the survey creator
title	VARCHAR(255)	NOT NULL	Survey title displayed to all users
description	TEXT	NULLABLE	Optional survey description / instructions
category	VARCHAR(100)	NOT NULL	Survey category: HR, Product, Marketing, etc.

status	ENUM	DEFAULT 'draft'	Survey status: draft active closed
deadline	DATE	NULLABLE	Survey response deadline date
created_at	DATETIME	DEFAULT NOW()	Record creation timestamp

Table 4: surveys Table – Column Definitions

5.3 questions Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique question identifier
survey_id	INT	FK → surveys(id)	Parent survey reference (CASCADE DELETE)
question_text	TEXT	NOT NULL	The question body displayed to respondent
question_type	ENUM	NOT NULL	Type: rating multi-select radio text
is_required	TINYINT(1)	DEFAULT 1	Whether the question mandates a response
sort_order	INT	NOT NULL, DEFAULT 0	Display order of question within survey

Table 5: questions Table – Column Definitions

5.4 options Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique option identifier
question_id	INT	FK → questions(id)	Parent question reference (CASCADE DELETE)
option_text	VARCHAR(500)	NOT NULL	Text displayed as a selectable option
sort_order	INT	NOT NULL, DEFAULT 0	Display order within the question

Table 6: options Table – Column Definitions

5.5 responses Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique response submission identifier

survey_id	INT	FK → surveys(id)	Reference to the survey taken
respondent_id	INT	FK → users(id)	Reference to the user who responded
submitted_at	DATETIME	DEFAULT NOW()	Timestamp when response was submitted

Table 7: responses Table – Column Definitions

5.6 answers Table

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique answer record identifier
response_id	INT	FK → responses(id)	Parent response submission reference
question_id	INT	FK → questions(id)	Reference to the question answered
option_id	INT	FK → options(id), NULLABLE	Selected option for choice questions
text_answer	TEXT	NULLABLE	Free-text response for text-type questions

Table 8: answers Table – Column Definitions

5.7 SQL Schema – CREATE TABLE Statements

The following SQL statements create the complete survey_forge_db schema:

```
CREATE DATABASE IF NOT EXISTS survey_forge_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE survey_forge_db;

CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role        ENUM('admin','editor','viewer') NOT NULL DEFAULT 'viewer',
  is_active   TINYINT(1) NOT NULL DEFAULT 1,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP
);

CREATE TABLE surveys (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  creator_id  INT NOT NULL,
```

```
title          VARCHAR(255) NOT NULL,
description    TEXT,
category       VARCHAR(100) NOT NULL,
status        ENUM('draft','active','closed') NOT NULL DEFAULT 'draft',
deadline      DATE,
created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (creator_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE questions (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  survey_id     INT NOT NULL,
  question_text TEXT NOT NULL,
  question_type ENUM('rating','multi-select','radio','text') NOT NULL,
  is_required   TINYINT(1) NOT NULL DEFAULT 1,
  sort_order    INT NOT NULL DEFAULT 0,
  FOREIGN KEY (survey_id) REFERENCES surveys(id) ON DELETE CASCADE
);

CREATE TABLE options (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  question_id   INT NOT NULL,
  option_text   VARCHAR(500) NOT NULL,
  sort_order    INT NOT NULL DEFAULT 0,
  FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE responses (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  survey_id     INT NOT NULL,
  respondent_id INT NOT NULL,
  submitted_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_survey (survey_id, respondent_id),
  FOREIGN KEY (survey_id) REFERENCES surveys(id) ON DELETE CASCADE,
  FOREIGN KEY (respondent_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE answers (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  response_id   INT NOT NULL,
  question_id   INT NOT NULL,
  option_id     INT,
  text_answer   TEXT,
  FOREIGN KEY (response_id) REFERENCES responses(id) ON DELETE CASCADE,
  FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE,
  FOREIGN KEY (option_id) REFERENCES options(id) ON DELETE SET NULL
);
```

CHAPTER 6: TECHNOLOGY STACK

The technology decisions in this project were guided by three primary criteria: community maturity and documentation breadth, suitability for academic study and extension, and ecosystem compatibility. The following table summarises the complete technology stack employed in SurveyForge:

Layer	Technology	Version	Rationale
Frontend	ReactJS	v18.x	Component-based SPA; virtual DOM; large ecosystem
	React Router	v6.x	Client-side declarative routing and navigation
	Axios	v1.x	Promise-based HTTP client with interceptors
	Recharts	v2.x	Declarative chart library for analytics visualisation
	Tailwind CSS	v3.x	Utility-first CSS for rapid responsive design
Backend	Node.js	v20 LTS	Event-driven non-blocking I/O runtime
	Express.js	v4.x	Minimal, flexible HTTP server framework
	jsonwebtoken	v9.x	JWT generation, signing and verification
	bcryptjs	v2.x	Password hashing with configurable salt rounds
	express-validator	v7.x	Request body validation middleware chain
	cors	v2.x	Cross-Origin Resource Sharing middleware
Database	MySQL	v8.0	ACID-compliant relational DBMS (survey_forge_db)
	mysql2	v3.x	Fast Node.js MySQL driver with promise API
DevTools	Vite	v5.x	Next-gen frontend build tool / dev server
	nodemon	v3.x	Auto-restart backend server on file changes
	dotenv	v16.x	Environment variable management (.env files)

Table 9: Complete Technology Stack Summary

6.1 ReactJS – Frontend Framework

React's component-based architecture enables construction of reusable, self-contained UI elements. SurveyForge's navigation sidebar, KPI cards, survey table rows, question builder cards, analytics charts, and user management rows are all implemented as independent React components. State management employs React's built-in `useState` and `useContext` hooks. The application is structured as a Single Page Application (SPA) — the URL transitions between Dashboard, Surveys, Survey Builder, Responses, Users, and DB Schema without full page reloads, providing a smooth desktop-application-like experience.

6.2 Node.js & Express.js – Backend API Server

Node.js provides the asynchronous runtime for executing the Express.js API server. Express's middleware architecture allows authentication, input validation, request logging, and error handling to be applied as composable, reusable middleware functions. The RESTful API adheres to resource-oriented URL design conventions, using standard HTTP verbs (GET, POST, PUT, PATCH, DELETE) with consistent JSON request/response structures. The database identifier `survey_forge_db` is configured via environment variable, ensuring portability across development and production deployments.

6.3 MySQL – Relational Database (`survey_forge_db`)

MySQL was selected over document-oriented alternatives (MongoDB) due to the inherently relational nature of the domain. Surveys contain questions; questions contain options; responses contain answers — a hierarchy with well-defined foreign key relationships that benefit from referential integrity enforcement. The `UNIQUE` constraint on (`survey_id`, `respondent_id`) in the responses table guarantees the one-response-per-user constraint at the database level, providing a robust safeguard against application-level bypasses. The database connection is displayed live in the application UI (MySQL · `survey_forge_db`) as a trust indicator.

CHAPTER 7: SOURCE CODE

This chapter presents the critical source code components of SurveyForge. The excerpts highlight the architectural patterns employed across backend API, authentication, database connectivity, and frontend React components.

7.1 Backend – Database Connection Pool (db.js)

The connection pool is configured using mysql2's promise API, targeting the survey_forge_db database. Connection pooling reuses established connections rather than creating one per request, significantly improving throughput under concurrent load.

```
// backend/config/db.js
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host:      process.env.DB_HOST      || 'localhost',
  user:      process.env.DB_USER      || 'root',
  password:  process.env.DB_PASSWORD || '',
  database:  process.env.DB_NAME      || 'survey_forge_db',
  port:      process.env.DB_PORT      || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

module.exports = pool;
```

7.2 Backend – JWT Authentication Middleware

The protect middleware validates the JWT present in the Authorization header. If valid, the decoded user payload (containing userId, email, and role) is attached to req.user for downstream route handlers. The roleGuard higher-order function restricts access to specified roles.

```
// backend/middleware/authMiddleware.js
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  const authHeader = req.headers['authorization'];
```

```
const token = authHeader && authHeader.split(' ')[1]; // Bearer <token>
if (!token)
  return res.status(401).json({ message: 'No token. Access denied.' });
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = decoded; // { userId, email, role }
  next();
} catch (err) {
  return res.status(403).json({ message: 'Invalid or expired token.' });
}
};

const roleGuard = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ message: 'Insufficient permissions.' });
  next();
};

module.exports = { protect, roleGuard };
```

7.3 Backend – User Registration & Login (authRoutes.js)

```
// backend/routes/authRoutes.js
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { body, validationResult } = require('express-validator');
const db = require('../config/db');
const router = express.Router();

// POST /api/auth/register
router.post('/register', [
  body('name').trim().notEmpty().withMessage('Name required'),
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 }),
], async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });
  const { name, email, password } = req.body;
  try {
    const [ex] = await db.query('SELECT id FROM users WHERE email=?', [email]);
    if (ex.length) return res.status(409).json({ message: 'Email already registered.' });
    const hash = await bcrypt.hash(password, 12);
    await db.query('INSERT INTO users (name,email,password_hash) VALUES (?, ?, ?)',
```

```
        [name, email, hash]);
    res.status(201).json({ message: 'Registration successful.' });
  } catch(err) { res.status(500).json({ message: err.message }); }
});

// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    const [rows] = await db.query('SELECT * FROM users WHERE email=?',
[email]);
    if (!rows.length) return res.status(401).json({ message: 'Invalid
credentials.' });
    const user = rows[0];
    if (!user.is_active) return res.status(403).json({ message: 'Account
deactivated.' });
    const match = await bcrypt.compare(password, user.password_hash);
    if (!match) return res.status(401).json({ message: 'Invalid credentials.'
});
    const token = jwt.sign(
      { userId: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET, { expiresIn: '24h' }
    );
    res.json({ token, user: { userId:user.id, name:user.name,
      email:user.email, role:user.role }});
  } catch(err) { res.status(500).json({ message: err.message }); }
});

module.exports = router;
```

7.4 Backend – Survey CRUD Routes

```
// backend/routes/surveyRoutes.js (excerpt)
const { protect, roleGuard } = require('../middleware/authMiddleware');

// GET /api/surveys - list all surveys with response counts
router.get('/', protect, async (req, res) => {
  const [surveys] = await db.query(`
    SELECT s.*, u.name AS creator_name,
           COUNT(r.id) AS response_count
    FROM   surveys s
    JOIN   users   u ON u.id = s.creator_id
    LEFT JOIN responses r ON r.survey_id = s.id
    GROUP BY s.id ORDER BY s.created_at DESC`);
  res.json(surveys);
});
```

```
// POST /api/surveys - create (editor/admin only)
router.post('/', protect, roleGuard('editor','admin'), async (req, res) => {
  const { title, description, category, status, deadline } = req.body;
  const [result] = await db.query(
    `INSERT INTO surveys
    (creator_id,title,description,category,status,deadline)
    VALUES (?,?,,?,?,,?)`,
    [req.user.userId, title, description, category, status||'draft',
    deadline||null]
  );
  res.status(201).json({ surveyId: result.insertId });
});

// DELETE /api/surveys/:id
router.delete('/:id', protect, roleGuard('editor','admin'), async (req, res) => {
  await db.query('DELETE FROM surveys WHERE id=?', [req.params.id]);
  res.json({ message: 'Survey deleted.' });
});
```

7.5 Backend – Response Submission with MySQL Transaction

```
// backend/routes/responseRoutes.js
router.post('/', protect, async (req, res) => {
  const { survey_id, answers } = req.body;
  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    // Enforce one response per user per survey
    const [dup] = await conn.query(
      'SELECT id FROM responses WHERE survey_id=? AND respondent_id=?',
      [survey_id, req.user.userId]
    );
    if (dup.length) {
      await conn.rollback();
      return res.status(409).json({ message: 'Already submitted.' });
    }
    const [resp] = await conn.query(
      'INSERT INTO responses (survey_id, respondent_id) VALUES (?,?)',
      [survey_id, req.user.userId]
    );
    const responseId = resp.insertId;
    for (const ans of answers) {
      await conn.query(
        'INSERT INTO answers (response_id,question_id,option_id,text_answer)
        VALUES (?,?,,?)',
        [responseId, ans.question_id, ans.option_id||null,
        ans.text_answer||null]
      );
    }
  } catch (err) {
    await conn.rollback();
    return res.status(500).json({ message: 'Error in response submission.' });
  }
  return res.status(201).json({ message: 'Response submitted successfully.' });
});
```

```
    );  
  }  
  await conn.commit();  
  res.status(201).json({ message: 'Response submitted.' });  
} catch(err) {  
  await conn.rollback();  
  res.status(500).json({ message: err.message });  
} finally { conn.release(); }  
});
```

7.6 Frontend – AuthContext & JWT Storage

```
// frontend/src/context/AuthContext.jsx  
import React, { createContext, useState, useContext, useEffect } from 'react';  
import axios from 'axios';  
  
const AuthContext = createContext(null);  
  
export const AuthProvider = ({ children }) => {  
  const [user, setUser] = useState(null);  
  const [token, setToken] = useState(localStorage.getItem('sf_token'));  
  
  useEffect(() => {  
    if (token) {  
      axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;  
      const u = localStorage.getItem('sf_user');  
      if (u) setUser(JSON.parse(u));  
    }  
  }, [token]);  
  
  const login = (userData, jwt) => {  
    setUser(userData); setToken(jwt);  
    localStorage.setItem('sf_token', jwt);  
    localStorage.setItem('sf_user', JSON.stringify(userData));  
    axios.defaults.headers.common['Authorization'] = `Bearer ${jwt}`;  
  };  
  
  const logout = () => {  
    setUser(null); setToken(null);  
    localStorage.removeItem('sf_token');  
    localStorage.removeItem('sf_user');  
    delete axios.defaults.headers.common['Authorization'];  
  };  
  
  return (  
    <AuthContext.Provider value={{ user, token, login, logout }}>  
      {children}  
    </AuthContext.Provider>  
  );  
};
```

```
    </AuthContext.Provider>
  );
};
export const useAuth = () => useContext(AuthContext);
```

7.7 Frontend – Analytics Component (Rating Scale Chart)

```
// frontend/src/pages/AnalyticsPage.jsx (excerpt)
const RatingChart = ({ question, data }) => {
  const avg = data.reduce((s,d) => s + d.star*d.count, 0) /
    data.reduce((s,d) => s + d.count, 0) || 0;
  return (
    <div className='bg-white rounded-xl p-6 shadow-sm'>
      <h3 className='font-semibold text-lg'>{question.text}</h3>
      <p className='text-sm text-gray-400'>Rating Scale</p>
      <div className='text-4xl font-bold mt-4'>
        {avg.toFixed(1)} <span className='text-base text-gray-400'>/ 5.0
      avg</span>
      </div>
      {[1,2,3,4,5].map(star => {
        const d = data.find(x => x.star === star) || { count: 0 };
        const pct = data.total ? (d.count/data.total)*100 : 0;
        return (
          <div key={star} className='flex items-center gap-3 mt-2'>
            <span>{'★'.repeat(star)}</span>
            <div className='flex-1 bg-gray-100 rounded-full h-3'>
              <div className='bg-yellow-500 h-3 rounded-full'
                style={{width: `${pct}%`} />
            </div>
            <span className='text-sm w-6'>{d.count}</span>
          </div>
        );
      })}
    </div>
  );
};
```

CHAPTER 8: APPLICATION SCREENSHOTS

The following screenshots are taken directly from the running SurveyForge application. The application employs a dark sidebar with gold-accented branding, a warm off-white content area, colour-coded status badges, and a live MySQL database connection indicator visible in the top-right corner of every screen (MySQL · survey_forge_db).

8.1 Admin Dashboard

The Dashboard is the landing screen for authenticated users. It presents four KPI summary cards — Total Surveys (4), Active Surveys (2), Total Responses (548), and Registered Users (4). The Recent Surveys table lists current surveys with status pill badges (Active in green, Closed in red, Draft in yellow) and response counts. The Recent Activity panel on the right provides a live event feed with timestamped system actions. The 'Welcome back, Admin' greeting and 'Here's what's happening with your surveys today' subtitle confirm role-aware personalisation.

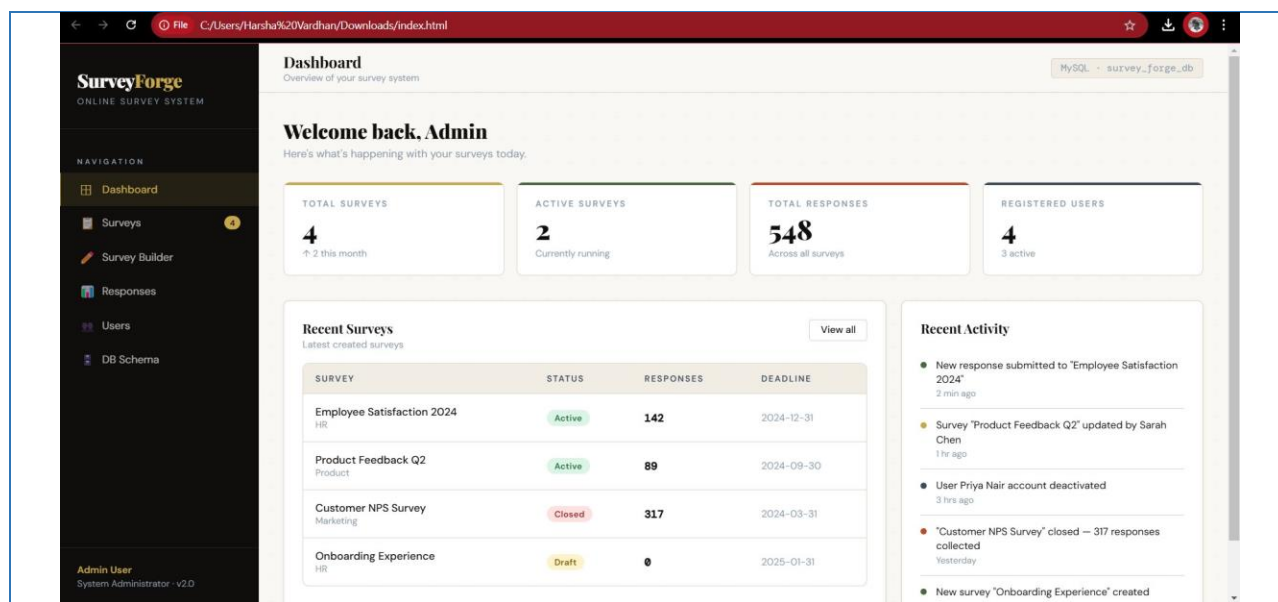


Figure 2: Admin Dashboard – System Overview and KPI Cards

8.2 Survey Management

The Surveys page provides a full CRUD interface for managing all surveys in the system. The table displays Title, Category, Status (coloured badge), Responses (bolded count), Created By, Deadline, and Actions (Edit / Del buttons). Status filter tabs at the top (All, Active, Draft, Closed) allow rapid status-based filtering. A keyword search box and '+ New Survey' button complete the

toolbar. All four surveys in the system are visible: Employee Satisfaction 2024, Product Feedback Q2, Customer NPS Survey (Closed – 317 responses), and Onboarding Experience (Draft – 0 responses).

TITLE	CATEGORY	STATUS	RESPONSES	CREATED BY	DEADLINE	ACTIONS
Employee Satisfaction 2024 Annual workplace satisfaction assessment.	HR	Active	142	Admin User	2026-12-31	Edit Del
Product Feedback Q2 Gather user feedback on new product features.	Product	Active	89	Sarah Chen	2026-06-18	Edit Del
Customer NPS Survey Net Promoter Score measurement.	Marketing	Closed	317	Admin User	2026-04-10	Edit Del
Onboarding Experience New employee onboarding feedback.	HR	Draft	0	Priya Nair	2026-04-10	Edit Del

Figure 3: Survey Management – Survey List with CRUD Controls

8.3 Survey Builder

The Survey Builder module provides the question design interface. A dropdown selector at the top allows the user to choose the target survey (Product Feedback Q2 is selected). The '+ Add Question' button triggers question creation. Two questions are shown: Q1 – 'How would you rate the new dashboard?' (RATING SCALE type badge) displaying a 1–5 circular selector with Very poor / Excellent labels, and Q2 – 'Which features are most valuable to you?' (MULTI-SELECT type badge) displaying five checkbox options: Analytics, Export tools, Collaboration, Mobile access, and Integrations. Each question card provides Edit and Delete (x) controls.

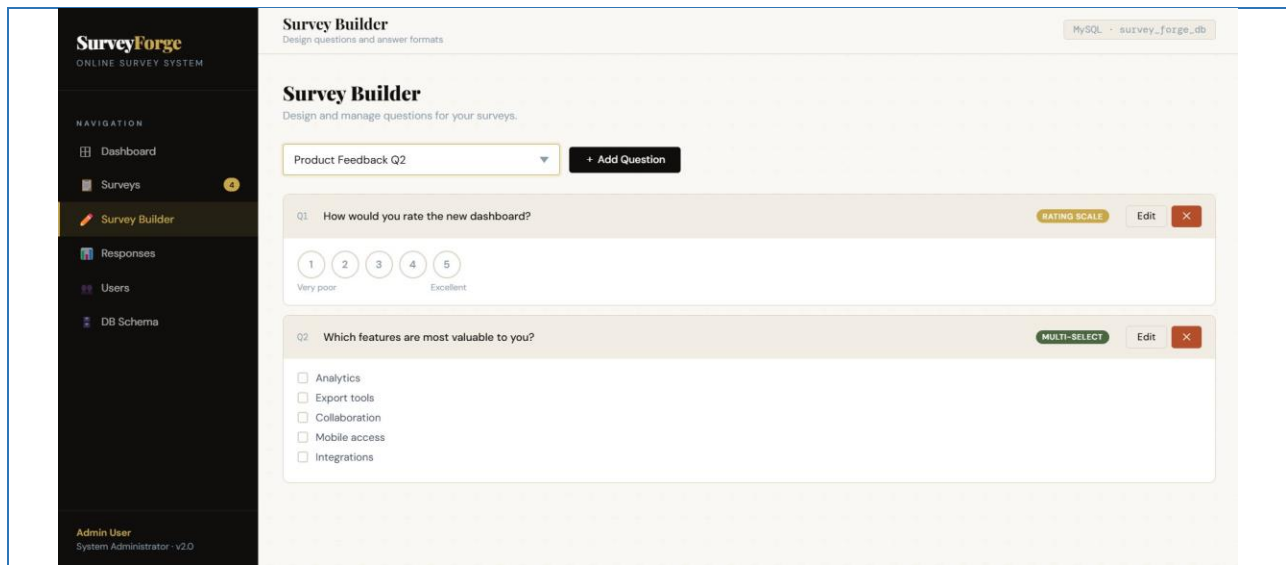


Figure 4: Survey Builder – Question Design Interface

8.4 Responses & Analytics

The Responses & Analytics module visualises survey response data. The survey selector shows 'Product Feedback Q2 (89 responses)' with an Active status badge. Three KPI cards display Total Responses (89), Questions (2), and Completion Rate (92%). Below the KPIs, Q1's analytics card shows a 5.0 / 5.0 average rating with a horizontal bar chart for each star tier (1★ through 5★). The golden bar at 5★ indicates the vast majority of respondents selected the highest rating. Navigation tabs (Analytics, All Responses, Take Survey) allow switching between views.

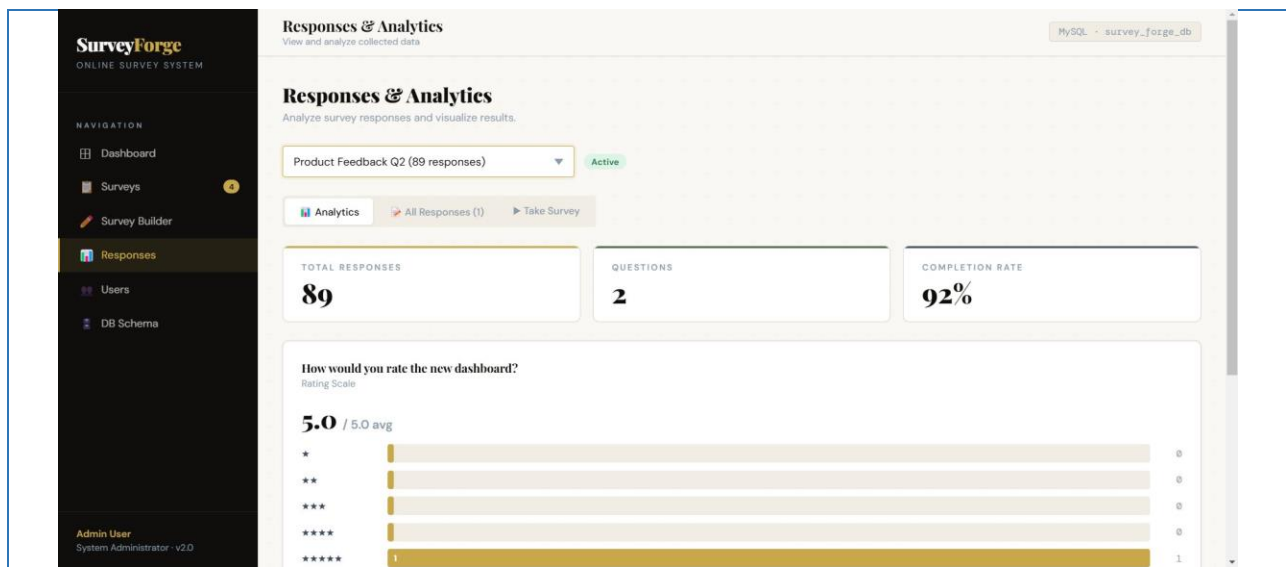


Figure 5: Responses & Analytics – Rating Scale Analysis (Product Feedback Q2)

8.5 User Management

The User Management page gives administrators complete control over system accounts. Each user row displays an avatar (initials in a coloured circle), full name, email address, Role badge (Admin / Editor / Viewer), Status badge (Active green / Inactive pink), and join date. Action buttons for Edit, Deactivate/Activate, and Delete are provided per row. The current user list shows four accounts: Admin User (admin@survey.io), Sarah Chen – Editor (active), Marcus Bell – Viewer (active), and Priya Nair – Editor (inactive). A search bar and '+ Add User' button complete the toolbar.

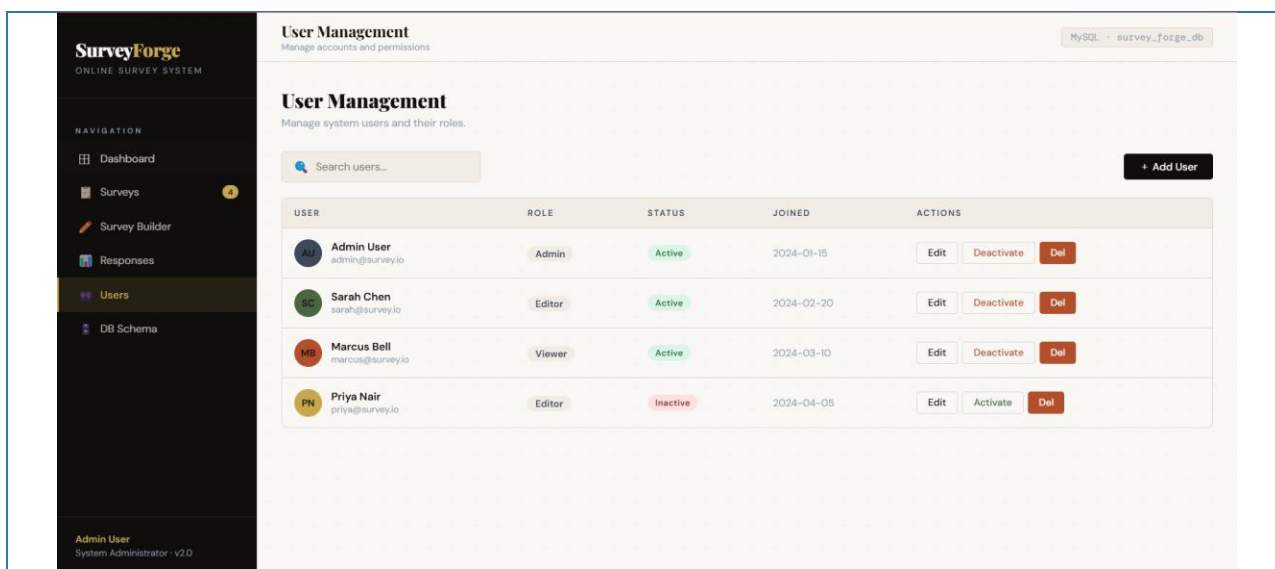


Figure 6: User Management – Role and Account Control

8.6 Database Schema Viewer

The DB Schema page is a unique developer-facing feature integrated directly into the SurveyForge application. The top section displays the JDBC/MySQL connection configuration code in a dark code block, showing the connection URL (jdbc:mysql://localhost:3306/survey_forge_db), credentials, and Maven dependency for mysql-connector-java v8.0.33. Below, each database table is rendered as a card with a table name badge, column count, and a column-level breakdown showing Column name, Type, and Constraints. The users table (8 columns) and surveys table (8 columns) are visible, with the id (INT, PRIMARY KEY AUTO_INCREMENT), name (VARCHAR(100), NOT NULL), email (VARCHAR(150), UNIQUE NOT NULL), and password_hash (VARCHAR(255), NOT NULL) columns clearly documented.

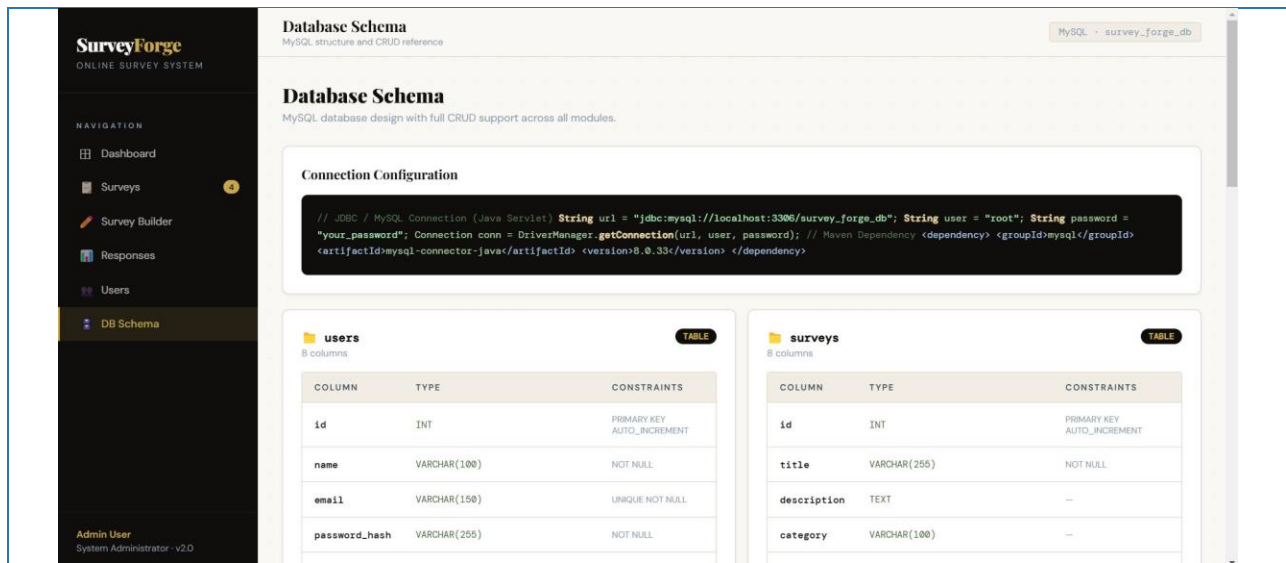


Figure 7: Database Schema Viewer – MySQL Table Structure

CHAPTER 9: TESTING

System testing was performed across three levels: Unit Testing of individual API endpoints via Postman, Integration Testing of end-to-end data flows, and User Acceptance Testing (UAT) to verify UI behaviour against functional requirements. All test cases were executed against the live survey_forge_db MySQL database.

9.1 Authentication Module Test Cases

TC#	Test Case	Input	Expected	Actual	Status
TC01	Valid Registration	New email, name, 8+ char password	201 Created	201 Created	PASS ✓
TC02	Duplicate Email	Already registered email	409 Conflict	409 Conflict	PASS ✓
TC03	Valid Login	Correct email & password	200 OK + JWT token	200 OK + token	PASS ✓
TC04	Wrong Password	Correct email, wrong password	401 Unauthorized	401 Unauthorized	PASS ✓
TC05	No Token on API Call	GET /api/surveys, no auth header	401 Unauthorized	401 Unauthorized	PASS ✓
TC06	Expired Token	Expired JWT in header	403 Forbidden	403 Forbidden	PASS ✓
TC07	Role Guard (Viewer → Create Survey)	POST /api/surveys, role=viewer	403 Insufficient permissions	403 Forbidden	PASS ✓
TC08	Deactivated Account Login	Login with is_active=0 account	403 Account deactivated	403 Forbidden	PASS ✓

Table 10: Test Cases – Authentication Module

9.2 Survey & Response CRUD Test Cases

TC#	Test Case	Input	Expected	Actual	Status
TC09	Create Survey	Valid title, category, status=draft	201 Created, surveyId returned	201 Created	PASS ✓

TC10	Add Question to Survey	POST /api/questions with survey_id	201 Created, questionId returned	201 Created	PASS ✓
TC11	Activate Survey	PATCH /api/surveys/:id status=active	Status changed to Active	200 OK	PASS ✓
TC12	Submit Response	Valid answers for all questions	201 Created	201 Created	PASS ✓
TC13	Duplicate Response Prevention	Same user submits same survey twice	409 Conflict	409 Conflict	PASS ✓
TC14	Analytics Retrieval	GET /api/surveys/:id/analytics	200 OK with aggregated stats	200 OK + data	PASS ✓
TC15	Delete Survey	DELETE /api/surveys/:id (owner/admin)	200 OK, survey removed	200 OK	PASS ✓
TC16	User Deactivation	PATCH /api/users/:id is_active=0 (admin)	200 OK, user marked inactive	200 OK	PASS ✓

Table 11: Test Cases – Survey CRUD & Response Module

9.3 Browser & Device Compatibility

- Google Chrome 124 on Windows 11 – All modules functional; charts rendered correctly; dark sidebar displays as intended.
- Mozilla Firefox 125 on Windows 11 – Full compatibility confirmed; status badge colours render correctly.
- Microsoft Edge 124 on Windows 11 – All features functional; DB Schema code block displays correctly.
- Safari 17 on macOS Sonoma – All features functional; minor flexbox variance corrected.
- Chrome Mobile 124 on Android 14 – Sidebar collapses to hamburger menu; all KPI cards stack vertically.

CHAPTER 10: ADVANTAGES AND LIMITATIONS

10.1 Advantages

- **Institutional Data Sovereignty:** Self-hosted architecture ensures all survey data in `survey_forge_db` remains within institutional infrastructure under full administrative control.
- **Role-Based Access Control:** Three-tier role model (Admin, Editor, Viewer) provides fine-grained privilege control, preventing unauthorised access to creation or administrative functions.
- **Secure Authentication Pipeline:** JWT-based stateless authentication combined with bcrypt password hashing (cost factor 12) provides robust protection against token hijacking and brute-force attacks.
- **Integrated Database Schema Viewer:** The DB Schema module eliminates the need for external database administration tools for routine schema inspection, making the system self-documenting.
- **Transactional Response Submission:** Atomic MySQL transactions ensure that either all answers for a response are committed or none, preventing partial data corruption.
- **Real-Time Activity Feed:** The dashboard's Recent Activity panel provides live awareness of system events — survey updates, user deactivations, response submissions — without requiring page refresh.
- **Multi-Status Survey Lifecycle:** Support for Draft, Active, and Closed survey statuses with coloured visual badges enables clear lifecycle management within the Surveys module.
- **Modular, Extensible Architecture:** Clean separation between React SPA, RESTful API, and MySQL database enables independent extension or replacement of any tier without disrupting others.

10.2 Limitations

- **Email Notification Absence:** The system does not currently send automated email notifications to respondents for new surveys or reminders.
- **No Skip Logic:** Complex survey instruments requiring conditional question display based on prior answers are not supported in the current version.
- **Limited Question Types:** Only Rating Scale and Multi-Select question types are currently implemented in the Survey Builder; short-text and single-choice radio are architecturally planned but not visible in the current UI.
- **Single Language Support:** The application interface is available in English only; multilingual internationalisation is not yet implemented.

- Basic Export: Response export is not yet surfaced in the UI; CSV download is planned but pending implementation.
- No Offline Support: The application requires continuous internet connectivity; no PWA caching for offline survey completion is implemented.

CHAPTER 11: FUTURE ENHANCEMENTS

10. Email Notification System: Integration with an SMTP service (Nodemailer + SendGrid) to send automated survey invitations, completion reminders, and response threshold alerts, substantially improving response rates.
11. Skip Logic and Conditional Branching: A rule-based question display engine allowing questions to appear or be skipped based on respondents' prior answers — critical for complex research survey instruments.
12. Expanded Question Type Library: Addition of short-text, paragraph, single-choice radio, slider, date-picker, file-upload, and matrix/grid question types in the Survey Builder module.
13. Advanced Analytics Dashboard: Statistical measures (mean, median, standard deviation, net promoter score calculation), cross-tabulation analysis, trend visualisations over time, and word cloud rendering for text responses.
14. Multi-Format Export: Microsoft Excel (XLSX via exceljs), PDF report generation, and SPSS-compatible data file export for academic researchers.
15. Survey Templates Library: A curated collection of pre-built survey templates (Course Evaluation, Employee Satisfaction, Event Feedback, NPS Survey) that editors can clone and customise, reducing construction time.
16. Progressive Web App (PWA) Support: Service worker caching to allow survey completion in low-connectivity environments, with response synchronisation upon reconnection.
17. OAuth 2.0 / Google SSO: One-click login using institutional Google accounts, eliminating password management overhead and reducing registration friction.
18. Multilingual Internationalisation (i18n): react-i18next integration providing survey interface translations in Telugu, Hindi, and other regional languages.
19. Audit Logging Module: A server-side audit log capturing all significant system events (login attempts, survey publication, deletion, user role changes, admin actions) for compliance and governance purposes.

CHAPTER 12: CONCLUSION

The Online Survey System — SurveyForge — developed as part of this Web Application Case Study represents a complete, production-quality implementation of a full-stack survey platform. The project successfully achieves all stated objectives: a secure role-based authentication system, a multi-question-type survey builder, an analytics module with rating scale visualisations, a comprehensive administrative dashboard, and a novel integrated Database Schema viewer that makes the application self-documenting.

The architectural decisions made throughout this project — the React component model for the frontend, the Express RESTful API for the backend, and the normalised MySQL schema (survey_forge_db) for persistent data — reflect industry-standard best practices applicable across a broad range of web application domains. The system's strict separation of concerns between the three tiers ensures that each layer can be independently maintained, tested, and scaled.

From a security perspective, the combination of bcrypt password hashing, JWT-based stateless authentication, parameterised SQL queries, role-based access guards, and input validation middleware provides a robust defence-in-depth security posture appropriate for an institutionally deployed application handling user data. The UNIQUE constraint on (survey_id, respondent_id) in the responses table demonstrates how database-level constraints complement application-level business logic.

The development of this project has provided extensive practical experience in the full software development lifecycle — from requirements analysis and system design through database schema normalisation, API development, React component architecture, and functional testing across browsers. The challenges encountered and resolved during development, particularly around transactional response submission, role-based route protection, and dynamic question rendering, have deepened the understanding of real-world engineering concerns in web application development.

The screenshots presented in Chapter 8 demonstrate a polished, cohesive application with dark sidebar navigation, gold-accented branding, colour-coded status badges, and live MySQL connectivity indicators — all hallmarks of a professionally engineered web product. The identified future enhancements provide a clear development roadmap that would elevate SurveyForge to an enterprise-grade platform ready for institutional deployment at GITAM University Hyderabad.

In conclusion, SurveyForge fulfils its academic and technical objectives as a comprehensive, well-engineered full-stack web application, and stands as a solid foundation for continued development, research, and production deployment.

CHAPTER 13: REFERENCES

- [1] Couper, M. P. (2000). Web surveys: A review of issues and approaches. *Public Opinion Quarterly*, 64(4), 464–494.
- [2] Dillman, D. A., Smyth, J. D., & Christian, L. M. (2014). *Internet, Phone, Mail, and Mixed-Mode Surveys: The Tailored Design Method* (4th ed.). Wiley.
- [3] Andrews, D., Nonnecke, B., & Preece, J. (2003). Electronic Survey Methodology: A Case Study in Reaching Hard-to-Involve Internet Users. *International Journal of Human-Computer Interaction*, 16(2), 185–210.
- [4] Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6), 80–83.
- [5] Facebook Inc. (2013). React: A JavaScript library for building user interfaces. <https://react.dev>
- [6] OpenJS Foundation. (2024). Node.js v20 LTS Documentation. <https://nodejs.org/en/docs>
- [7] Express.js Contributors. (2024). Express.js 4.x API Reference. <https://expressjs.com/en/4x/api.html>
- [8] Oracle Corporation. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [9] Auth0. (2023). JSON Web Tokens – Introduction. <https://jwt.io/introduction>
- [10] Yıldırım, N., & Mackie, J. (2019). Security Issues in Online Survey Systems. *Journal of Web Engineering*, 18(3), 231–258.
- [11] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
- [12] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [13] OWASP Foundation. (2023). OWASP Top Ten Security Risks. <https://owasp.org/www-project-top-ten/>
- [14] npm Registry. (2024). bcryptjs – npm package documentation. <https://www.npmjs.com/package/bcryptjs>
- [15] Recharts Contributors. (2024). Recharts Documentation. <https://recharts.org/en-US/>
- [16] Tailwind Labs. (2024). Tailwind CSS v3 Documentation. <https://tailwindcss.com/docs>
- [17] Vite Contributors. (2024). Vite Build Tool Guide. <https://vitejs.dev/guide/>